

Proiectarea cu Microprocesoare
Curs 2

Andrei Voinescu

POLITEHNICA University of Bucharest

4 martie 2020

- Încărcarea programului
  - Cu ajutorul unui programator extern sau a unui bootloader, binarul de AVR trebuie să ajungă în memoria de program
- Lansarea în execuție
  - La resetarea microcontroller-ului, se pornește execuția începând cu adresa 0[1]
- Fire de execuție
  - Unul singur, dat de registrul `PC` (Program Counter)
- Interacțiune cu exteriorul
  - Doar prin instrucțiuni `in`/`out` (port-mapped I/O)

[1] diferă în cazul bootloader-ului

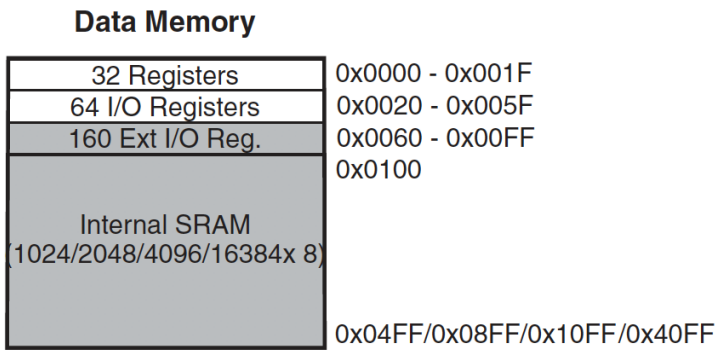
- Programul poate fi pus în memoria de program cu ajutorul unui In-System Programmer sau a unui debugger extern
- Alternativ, putem folosi un bootloader, un program care funcționează doar dintr-o parte a memoriei de program și care poate scrie în restul memoriei
  - Fiind arhitectură Harvard, în mod normal nu există instrucțiuni de acces la memoria de program pentru scriere
    - În loc de instrucțiuni de `ld` și `st` (load și store), avem `lpm` și `spm`

- Bootloaderul ocupă spațiu din memoria de program
  - Dacă bootloader-ul ocupă 2kB, rămân 30kB pentru programul nostru
- În același timp trebuie să conțină codul necesar pentru a obține programul nou
- Memoria de program este în tehnologie Flash, deci trebuie scrise datele în **pagini** (dimensiunea depinzând de chip)

- În C, foarte similar cu programarea pe PC
- Input/Output?
  - Nu avem consolă
  - Poate avem LEDuri conectate la microcontroller
  - Putem avea un LCD
  - Serială, etc.
- Variabile?
  - Le putem avea în RAM (2kB)
  - Le putem avea și în Flash (32kB, împreună cu codul)
- Pornire la reset (sau după bootloader)
- GCC este configurat pentru un sistem pe 8 biți (`int` și pointerii sunt pe 16 biți)
- Avem macrodefiniții pentru toate adresele relevante de I/O, compilatorul știe automat să folosească instrucțiunile dedicate pentru scrierile și citirile de la aceste adrese

```
#define PORTC *(volatile uint8_t *) (0x020)
```

- Cum clasificăm variabilele într-un mediu de programare pe PC în funcție de alocarea memoriei lor?
- Variabilele pot fi locale funcției → pe stivă
- Variabilele pot fi statice → secțiunea `.bss`
- Variabilele pot fi alocate dinamic → heap
  - Presupune adăugarea codului pentru alocator de memorie
  - Memory overhead pentru un dispozitiv care are foarte puțină memorie în general
- Constante! Le putem ține în memoria de program
- Stocare non-volatilă există și în EEPROM
  - Citire și scriere greoaie
  - 1kB de memorie
  - număr destul de limitat de scrieri (100 000x)
  - util pentru ce trebuie citit/scris doar o dată per restart



- 32 registre general purpose accesibile și ca memorie
- 64 registre I/O accesibile și cu `in`/`out`
- 160 de registre I/O extra
- 2048 de octeți de memorie SRAM

Hello, world!

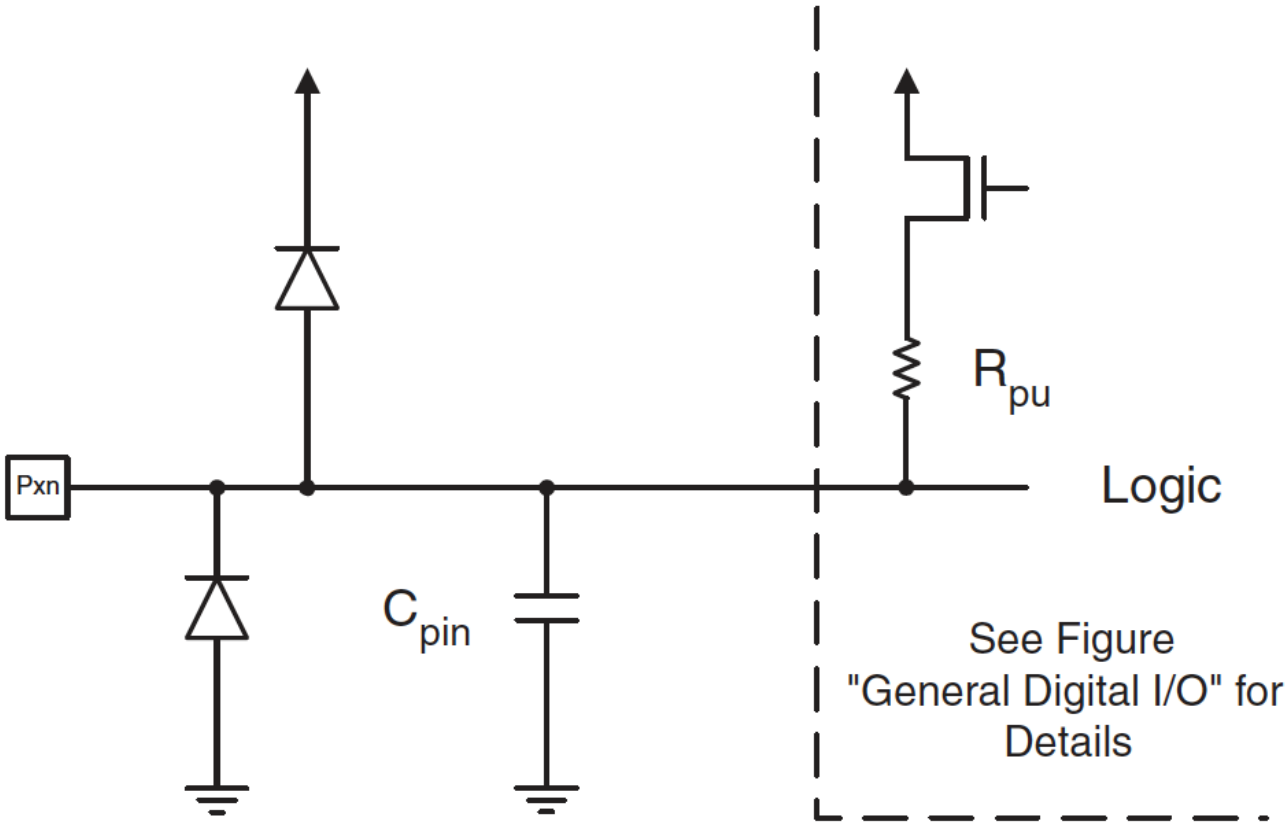
```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

int main()
{
    DDRC = (1 << PC4);


    while (1)
    {
        PORTC ^= (1 << PC4);
        _delay_ms(250);
    }
}
```

- Header-ul cu registrele I/O
- Frecvența de lucru (determinată de placă)
- Setarea pinului PC4 ca ieșire
- Alternarea stării pinului PC4 o dată la 250 de milisecunde

Un pin I/O


Pxn
C_{pin}
R_{pu}
Logic
See Figure
"General Digital I/O" for
Details



Configurarea unui PIN I/O

DDxn	PORTxn	PUD (in MCUCR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low.
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output Low (Sink)
1	1	X	Output	No	Output High (Source)

Cum arată rezultatul?

```
00000000 <__vectors>:
   0:   0c 94 3e 00     jmp 0x7c            ; 0x7c <__ctors_end>
                                            ; care face jump catre main dupa initializar
00000094 <main>:
  94:   80 e1           ldi r24, 0x10       ; (1 << 4)
  96:   87 b9           out 0x07, r24       ; scriere in DDRC
  98:   90 e1           ldi r25, 0x10       ; (1 << 4)
  9a:   88 b1           in  r24, 0x08       ; citire din PORTC
  9c:   89 27           eor r24, r25        ; r24 = r24 XOR r25
  9e:   88 b9           out 0x08, r24       ; scriere in PORTC
  a0:   2f ef           ldi r18, 0xFF       ; 255
  a2:   34 e3           ldi r19, 0x34       ; 52
  a4:   8c e0           ldi r24, 0x0C       ; 12
  a6:   21 50           subi    r18, 0x01   ; 1 ciclu
  a8:   30 40           sbci    r19, 0x00   ; 1 ciclu
  aa:   80 40           sbci    r24, 0x00   ; 1 ciclu
  ac:   e1 f7           brne    .-8         ; 0xa6 <main+0x12>
                                            ; 1 ciclu daca nu face conditia, 2 altfel
                                            ; practic se numara de la 0x0C34FF la 0
                                            ; 0x0C34FF * 5 cicli - 1 de iesire
  ae:   00 c0           rjmp    .+0         ; 0xb0 <main+0x1c>
  b0:   00 00           nop                 ; nop + rjmp = 3 cicli
  b2:   f3 cf           rjmp    .-26        ; 0x9a <main+0x6>
```

- Ce sunt întreruperile?
  - Secvențe de cod care se execută asicron cu restul programului, ca urmare a unui stimul
- Cum apar ele?
  - Factori interni (timere, terminarea unor operații lungi)
  - Factori externi (serială, butoane)
- Cum se tratează?
  - Rutine de tratare a întreruperii - RTI (en. ISR)

- Cum descriu complet un program aflat în funcționare?
  - Program Counter (PC)
  - Status Register (flag de carry, flag de zero - SREG)
  - Stiva curentă
  - Registre + memorie
- Ce face procesorul pentru a trata întreruperea?
  - Salvez **contextul**
  - Găsesc adresa rutinei de tratare
  - Execut rutina de tratare
  - Refac **contextul**

- La declanșarea unei întreruperi
  - PC+2 pe stivă
  - Se dezactivează bitul I din SREG
- La revenirea din întrerupere (RETI)
  - PC ia valoarea de pe stivă
  - Se activează bitul I din SREG

- Ce trebuie să fac?
- Activez toate întreruperile
  - Întreruperile pot fi dezactivate complet cu flag-ul **I** din `SREG` - status register
- Activez întreruperea în cauză - fiecare are propriul pin de enable
- Scriu funcția de ISR

```
ISR(USART0_RX_vect)
{
    ...
}
```

EIMSK – External Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x1D (0x3D)	–	–	–	–	–	INT2	INT1	IINT0	EIMSK
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 2:0 – INT2:0: External Interrupt Request 2 - 0 Enable**

When an INT2:0 bit is written to one and the I-bit in the Status Register (SREG) is set (one), the corresponding external pin interrupt is enabled. The Interrupt Sense Control bits in the External Interrupt Control Register, EICRA, defines whether the external interrupt is activated on rising or falling edge or level sensed. Activity on any of these pins will trigger an interrupt request even if the pin is enabled as an output. This provides a way of generating a software interrupt.



Exemplu de folosire întreruperi

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

ISR(INT0_vect)
{
    PORTC ^= (1 << PC4);
}

int main()
{
    DDRC = (1 << PC4);

    EIMSK |= (1 << INT0);

    sei();

    while (1);
}
```

- Enable la întreruperea pe care o dorim
- Enable la toate întreruperile `sei()`
- rutina de tratarea ISR
- Am păstrat bucla infinită din main!



Cum arată codul?

```
00000000 <__vectors>:
   0:   jmp     0x7c    ; 0x7c <__ctors_end>
   4:   jmp     0x94    ; 0x94 <__vector_1>
   8:   jmp     0x90    ; 0x90 <__bad_interrupt>
   c:   jmp     0x90    ; 0x90 <__bad_interrupt>
                        ...

00000094 <__vector_1>:
  94:   push    r1
  96:   push    r0
  98:   in      r0, 0x3f        ; 63
  9a:   push    r0
  9c:   eor     r1, r1
  9e:   push    r24
  a0:   push    r25
  a2:   in      r24, 0x08       ; 8
  a4:   ldi     r25, 0x10       ; 16
  a6:   eor     r24, r25
  a8:   out     0x08, r24       ; 8
  aa:   pop     r25
  ac:   pop     r24
  ae:   pop     r0
  b0:   out     0x3f, r0        ; 63
  b2:   pop     r0
  b4:   pop     r1
  b6:   reti
```

- Tabela de vectori de întrerupere este la început
- Orice intrare nedefinită din tabelă duce la un software restart
- Ce ar fi fost diferit dacă era altă întrerupere (nu `INT0`)?

De ce avem bucla infinită fără cod?

```
00000096 <main>:
  96:   ldi     r24, 0x10       ; 16
  98:   out     0x07, r24       ; 7
  9a:   sbi     0x1d, 0 ; 29
  9c:   sei
  9e:   rjmp    .-2
; 0x9e <main+0x8>

000000a0 <_exit>:
  a0:   cli

000000a2 <__stop_program>:
  a2:   rjmp    .-2
; 0xa2 <__stop_program>
```

- `cli` dezactivează întreruperile
- `rjmp .-2` este un loop pe loc, cu întreruperile dezactivate

```
ISR(INT0_vect, ISR_NAKED)
{
    PORTC |= (1 << PC4);
}

00000094 <__vector_1>:
  94:   sbi     0x08, 4 ; 8
```

- `ISR_NAKED` nu mai dezactivează întreruperile
- setarea unui bit pe 1 poate fi făcută cu o singură instrucțiune




```
#include <avr/interrupt.h>
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

volatile int flag = 0;

ISR(INT0_vect)
{
    flag = 1;
}

int main()
{
    LCD_init();

    EIMSK |= (1 << INT0);

    sei();

    while (1)
    {
        if (flag)
        {
            LCD_print("PC4_pressed");
        }
    }
}
```

- Compilatorul nu poate crea dependențe de variabile între `main` și întreruperi
- `volatile` instruiește compilatorul să nu optimizeze în jurul acelei variabile
- Ce s-ar fi întâmplat dacă nu puneam `volatile`?

Alte detalii de boot

```
0000007c <__ctors_end>:
  7c:   eor     r1, r1
  7e:   out     0x3f, r1        ; 63
  80:   ldi     r28, 0xFF       ; 255
  82:   ldi     r29, 0x08       ; 8
  84:   out     0x3e, r29       ; 62
  86:   out     0x3d, r28       ; 61
  88:   call    0x96    ; 0x96 <main>
  8c:   jmp     0xa0    ; 0xa0 <_exit>
```

- 0x3f este SREG
- 0x3e și 0x3d sunt SPH și SPL, stiva pe 16 biți
- de ce punem în stivă valoarea 0x08FF?



- Avem un singur thread
  - Main function/loop
  - Asincron putem executa rutine
- Poate exista un sistem de multi-threading în AVR?
  - Timer + salvare de context

```
void thread_1() { ... }
void thread_2() { ... }

/* O data la 10ms */
ISR(Timer)
{
  save_context(peek(task_queue));
  task_t current = pop(task_queue);
  push(task_queue, current);
  restore_context(peek(task_queue));
}

void save_context(task_t task)
{
  push_all_registers();
  task.stack = SP;
}
void restore_context(task_t task)
{
  SP = task.stack;
  pop_all_registers();
}
```

- Scheduler Round Robin
- Cuantă de timp fixă
- Stive diferite



- Ce ar trebui să luăm în calcul pentru thread-uri?
  - Acces la resurse
  - Acces la memorie
  - Timp discontinuu